Algorithms

Lecture 1
Time Complexity & Asymptotic Analysis

Prantik Paul [PNP]
Lecturer
Department of Computer Science and Engineering
BRAC University

Algorithm Definition

- A finite set of statements that guarantees an optimal solution in finite interval of time

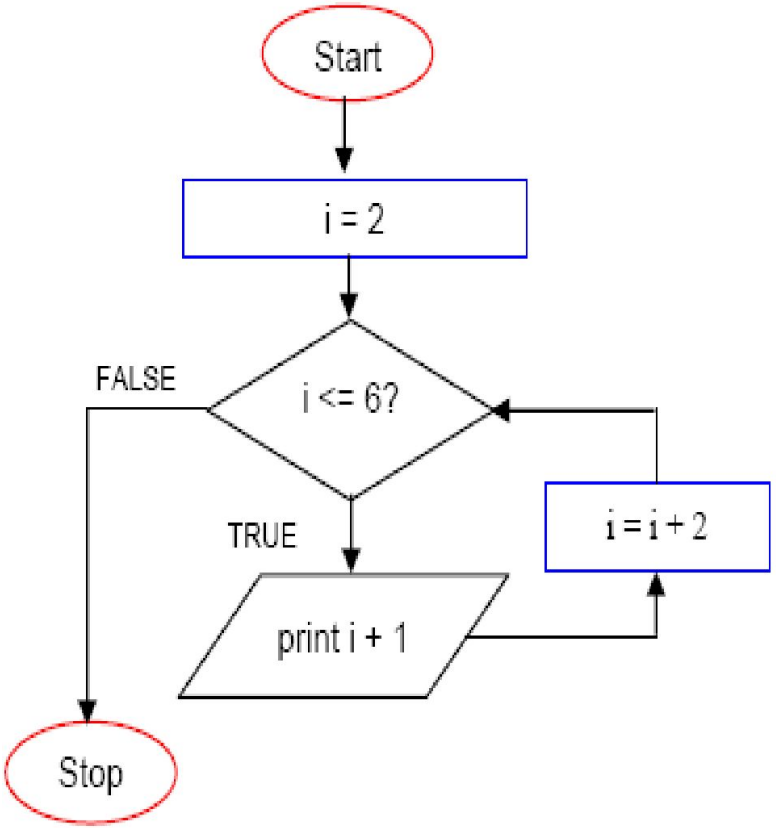
- Algorithmic thinking and problem solving skill are vital in making efficient solutions.

- The English word "ALGORITHM" derives from the Latin word AL-AL-KHWARIZMI'S name. He developed the concept of an algorithm in Mathematics, and thus sometimes being called the "Grandfather of Computer Science".

Glance of Algorithm

- An algorithm is a finite set of instructions or logic, written in order, to accomplish a certain predefined task.
- Algorithm is not the complete code or program
- Can be expressed either as an informal high level description as pseudocode or using a flowchart.

Glance of Algorithm



WHILE loop

- Do the loop body if the condition is true.
- Example: Get the sum of 1, 2, 3, …, 100.
  - Algorithm:
    - Set the number = 1
    - Set the total = 0
    - While (number <= 100)
      - total = total + number
      - number = number + 1
    - End While
    - Display total

Algorithm Specifications

- *Input* - Every Algorithm must take zero or more number of input values from external.
- *Output* - Every Algorithm must produce an output as result.
- *Definiteness* - Every statement/instruction in an algorithm must be clear and unambiguous (only one interpretation)
- *Finiteness* - For all different cases, the algorithm must produce result within a finite number of steps.
- *Effectiveness* - Every Instruction must be basic enough to be carried out and it also must be feasible.

Good Algorithms?

- Run in less time
- Consume less memory

But computational resources (time complexity) usually important

Analyzing Algorithms

- Predict the amount of resources required:
  - memory: how much space is needed?
  - computational time: how fast the algorithm runs?
- FACT: running time grows with the size of the input
- Input size (number of elements in the input)
  - Size of an array, polynomial degree, # of elements in a matrix, # of bits in the binary representation of the input, vertices and edges in a graph

Def: Running time = the number of primitive operations (steps) executed before termination

  - Arithmetic operations (+, -, *), data movement, control, decision making (*if, while*), comparison

Algorithm Analysis: Example

- *Alg.:* MIN (a[1], …, a[n])

  m ← a[1];
  for i ← 2 to n
  if a[i] < m
  then m ← a[i];

- **Running time**:
  - the number of primitive operations (steps) executed before termination

  $T(n) = 1$ [first step] + (n)[for loop] + $(n-1)$[if condition] + $(n-1)$[the assignment in then] = $3n - 1$

- Order (rate) of growth:
  - The leading term of the formula
  - Expresses the asymptotic behavior of the algorithm

Typical Running Time Functions

- 1 (constant running time):
  - Instructions are executed once or a few times
- logN (logarithmic)
  - A big problem is solved by cutting the original problem in smaller sizes, by a constant fraction at each step
- N (linear)
  - A small amount of processing is done on each input element
- N logN
  - A problem is solved by dividing it into smaller problems, solving them independently and combining the solution

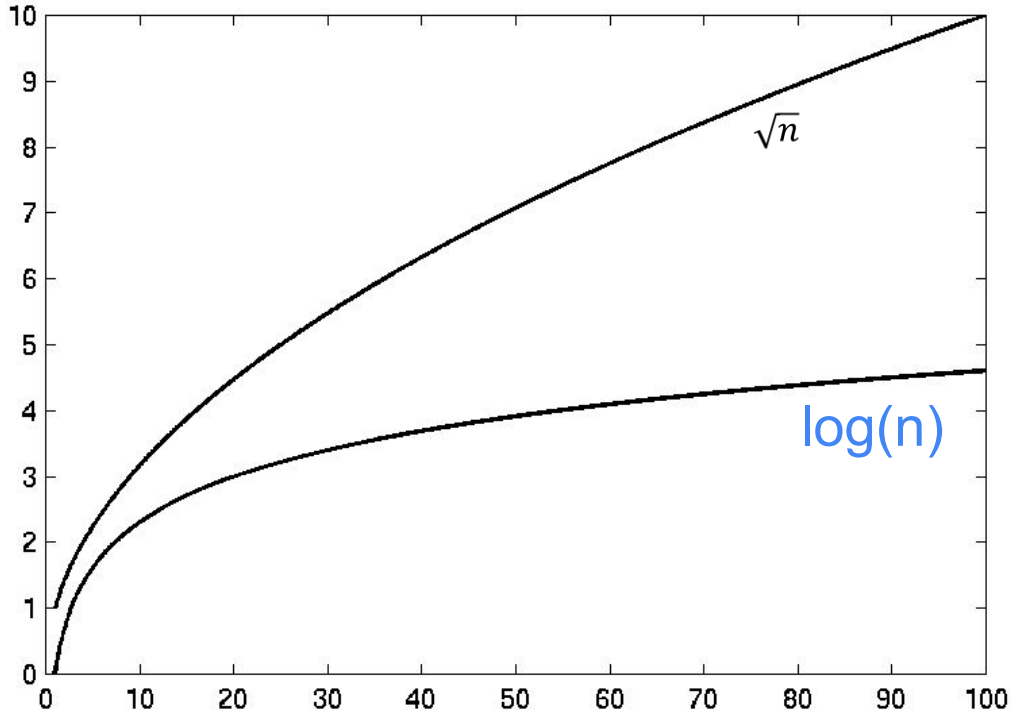
Typical Running Time Functions

- N^2 (quadratic)
  - Typical for algorithms that process all pairs of data items (double nested loops)
- N^3 (cubic)
  - Processing of triples of data (triple nested loops)
- N^K (polynomial)
- 2^N (exponential)
  - Few exponential algorithms are appropriate for practical use

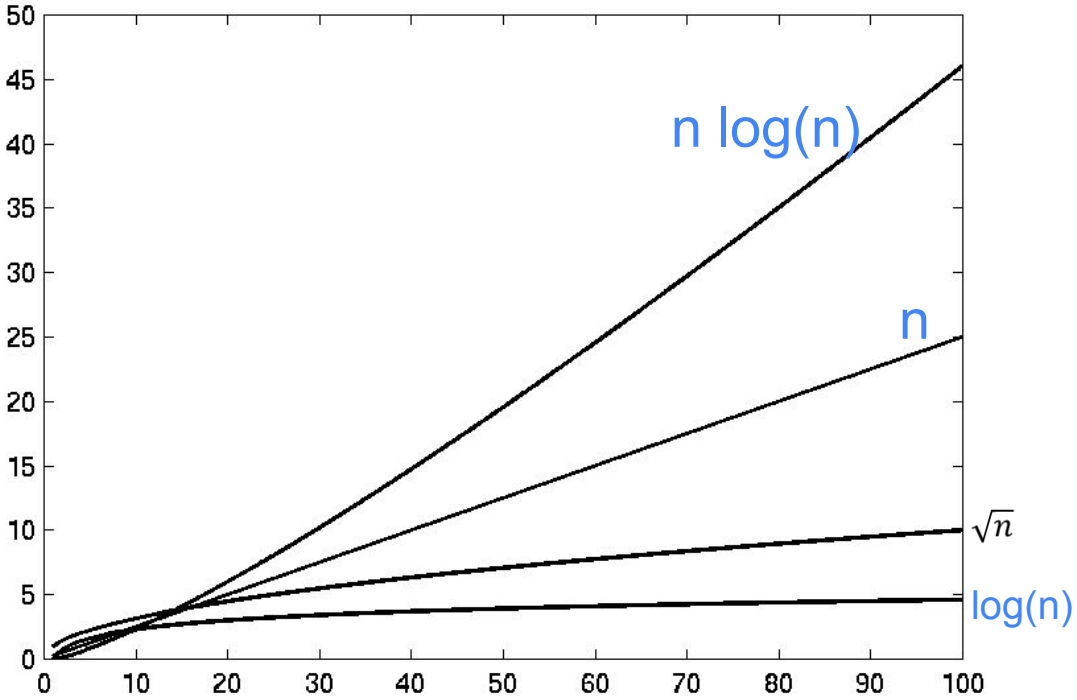
Growth of Functions

n	1	lgn	n	nlgn	n^2	n^3	2^n
1	1	0.00	1	0	1	1	2
10	1	3.32	10	33	100	1,000	1024
100	1	6.64	100	664	10,000	1,000,000	1.2×10^{30}
1000	1	9.97	1000	9970	1,000,000	10^9	1.1×10^{301}

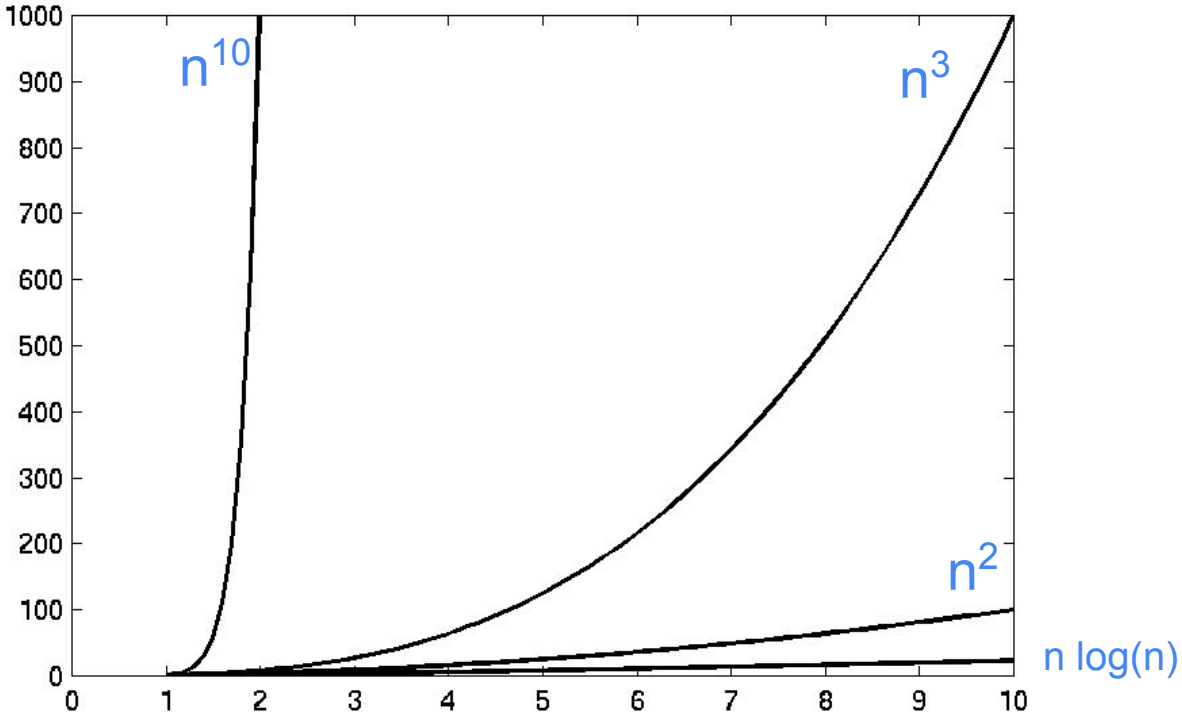
Complexity Graphs



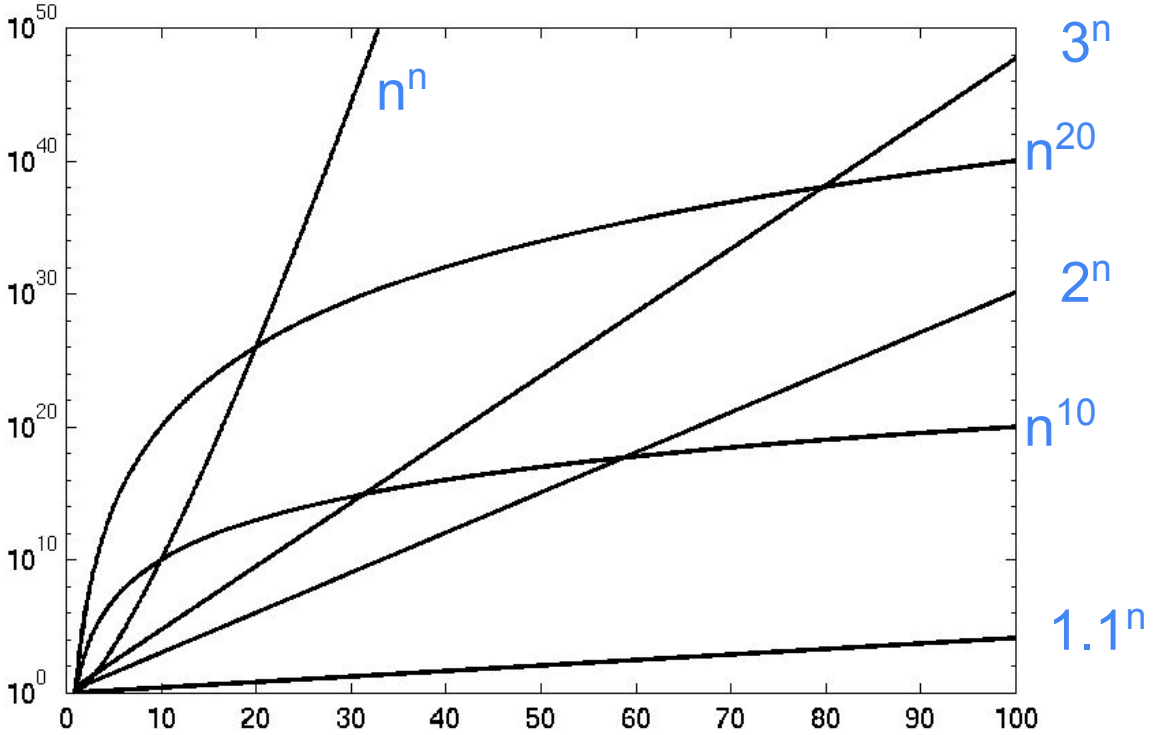
Complexity Graphs



Complexity Graphs



Complexity Graphs (log scale)



How do we compare algorithms?

- We need to define a number of objective measures.

(1) Compare execution times?

Not good: times are specific to a particular computer !!

(2) Count the number of statements executed?

Not good: number of statements vary with the programming language as well as the style of the individual programmer.

Ideal Solution

- Express running time as a function of the input size *n* (i.e., *f(n)*).
- Compare different functions corresponding to running times.
- Such an analysis is independent of machine time, programming style, etc.

Example

- Associate a "cost" with each statement.
- Find the "total cost“ by finding the total number of times each statement is executed.

Algorithm 1

	Cost
arr[0] = 0;	c_1
arr[1] = 0;	c_1
arr[2] = 0;	c_1
…	…
arr[N-1] = 0;	c_1

$c_1+c_1+\ldots+c_1 = c_1 \times N$

Algorithm 2

	Cost
for(i=0; i<N; i++)	c_2
arr[i] = 0;	c_1

$(N+1) \times c_2 + N \times c_1 = (c_2 + c_1) \times N + c_2$

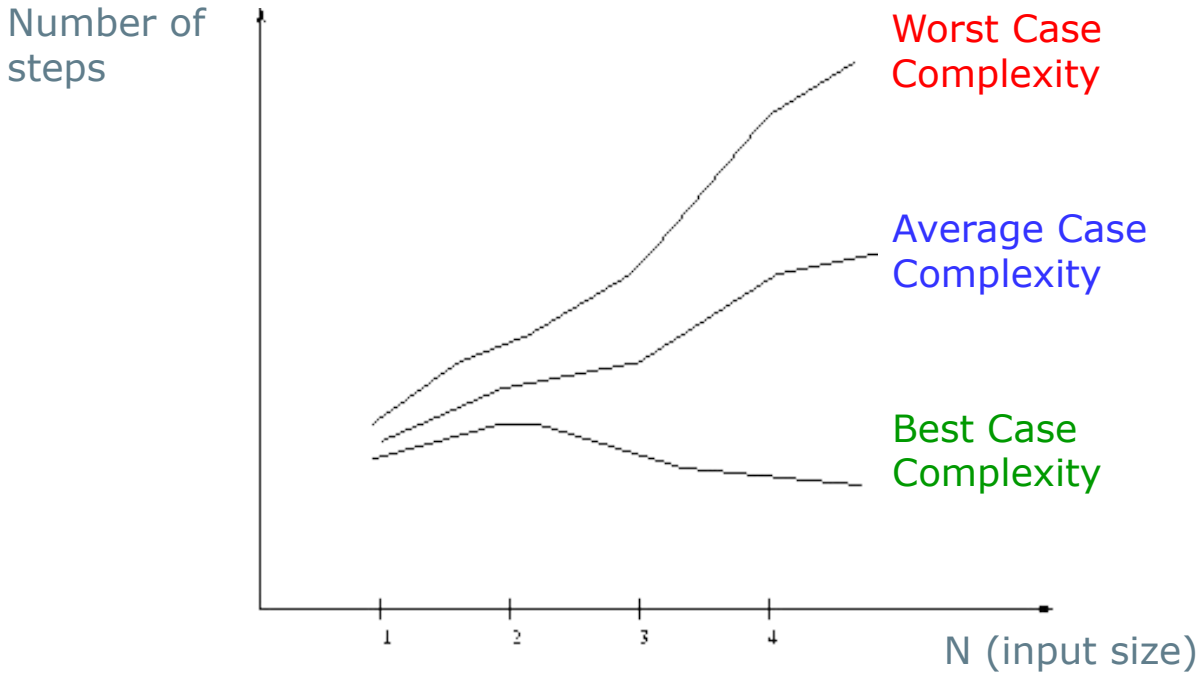
Another Example

- ***Algorithm 3*** *Cost*

```
sum = 0;                          c1
for(i=0; i<N; i++)                c2
   for(j=0; j<N; j++)             c2
      sum += arr[i][j];           c3
                                  ----------
```

$c_1 + c_2 \times (N+1) + c_2 \times N \times (N+1) + c_3 \times N^2$

Best, Worst, and Average Case Complexity

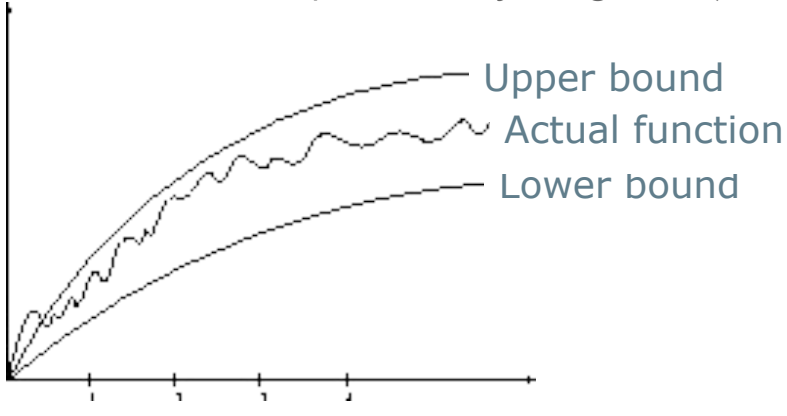


Algorithm Complexity

- Worst Case Complexity:
  - the function defined by the *maximum* number of steps taken on any instance of size *n*
- Best Case Complexity:
  - the function defined by the *minimum* number of steps taken on any instance of size *n*
- Average Case Complexity:
  - the function defined by the *average* number of steps taken on any instance of size *n*

Doing the Analysis

- It's hard to estimate the running time exactly
  - Best case depends on the input
  - Average case is difficult to compute
  - So we usually focus on worst case analysis
    - Easier to compute
    - Usually close to the actual running time
- Strategy: find a function (an equation) that, for large n, is an upper bound to the actual function (actual number of steps, memory usage, etc.)



Asymptotic Growth

When trying to decide whether an algorithm is efficient we are only interested in the value of its time complexity for large values of n, because for small values of n the running time of an algorithm is very small. (For example, in the previous table, for values of n smaller than 100 the running times are much smaller than 1 second. However, for n = 1 million, the running time of the algorithm is 173 days.)

Motivation for Asymptotic Analysis

- An *exact computation* of worst-case running time can be difficult
  - Function may have many terms:
    - $4n^2 - 3n \log n + 17.5 n - 43 n^{2/3} + 75$
- An *exact computation* of worst-case running time is unnecessary

Asymptotic Analysis

- To compare two algorithms with running times *f(n)* and *g(n)*, we need a **rough measure** that characterizes **how fast each function grows.**

- *Hint:* use *rate of growth*

- Compare functions in the limit, that is, **asymptotically!**

  (i.e., for large values of *n*)

Rate of Growth

- Consider the example of buying a car and a packet of biscuits:

  Cost: cost_of_car + cost_of_biscuit

  Cost ~ cost_of_car (approximation)

- The low order terms in a function are relatively insignificant for **large** n

  $n^4 + 100n^2 + 10n + 50 \sim n^4$

i.e., we say that $n^4 + 100n^2 + 10n + 50$ and n^4 have the same **rate of growth**

Classifying functions by their Asymptotic Growth Rates

1. O(g(n)), Big-Oh of g of n, the Asymptotic Upper Bound
2. Θ(g(n)), Theta of g of n, the Asymptotic Tight Bound
3. Ω(g(n)), Omega of g of n, the Asymptotic Lower Bound

Asymptotic Notation

- **O notation**: asymptotic “less than”:

  ➔ f(n)=O(g(n)) implies: f(n) “≤” g(n)

- **Ω notation**: asymptotic “greater than”:

  ➔ f(n)= Ω (g(n)) implies: f(n) “≥” g(n)

- **Θ notation**: asymptotic “equality”:

  ➔ f(n)= Θ (g(n)) implies: f(n) “=” g(n)

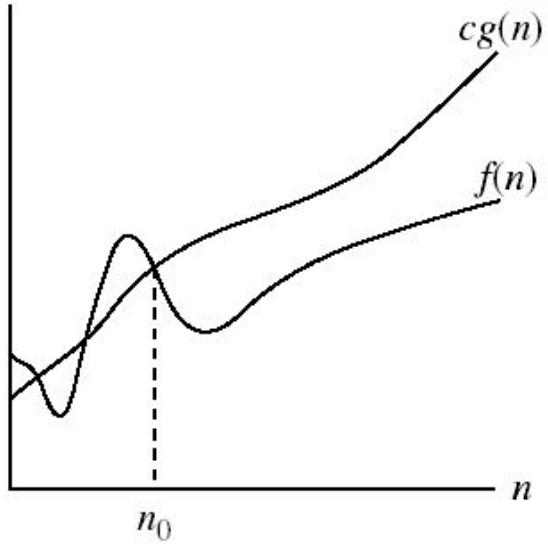
Big-O Notation

- We say $f_A(n)=30n+8$ is *order n*, or O (n)
  It is, at most, roughly *proportional* to *n*.
- $f_B(n)=n^2+1$ is *order* n^2, or $O(n^2)$. It is, at most, roughly proportional to n^2.
- In general, any $O(n^2)$ function is faster- growing than any $O(n)$ function.

Asymptotic notations

- *O-notation*

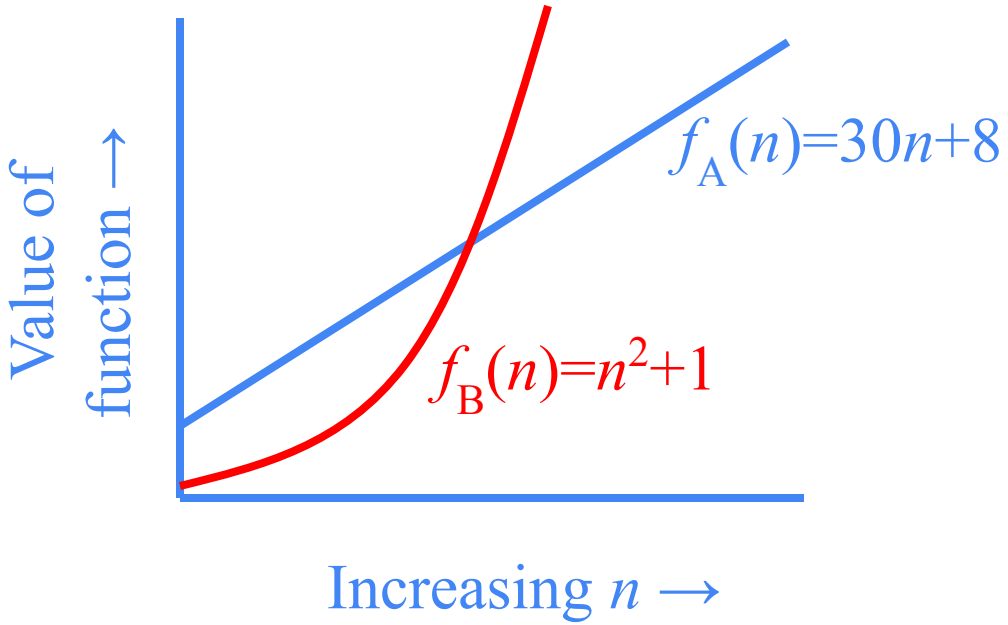
$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\}.$$



$g(n)$ is an ***asymptotic upper bound*** for $f(n)$.

Visualizing Orders of Growth

- On a graph, as you go to the right, a faster growing function eventually becomes larger...



More Examples …

- $n^4 + 100n^2 + 10n + 50$ is $O(n^4)$
- $10n^3 + 2n^2$ is $O(n^3)$
- $n^3 - n^2$ is $O(n^3)$
- constants
  - 10 is $O(1)$
  - 1273 is $O(1)$

Back to Our Example

Algorithm 1

	Cost
arr[0] = 0;	c_1
arr[1] = 0;	c_1
arr[2] = 0;	c_1
...	
arr[N-1] = 0;	c_1

$c_1+c_1+...+c_1 = c_1 \times N$

Algorithm 2

	Cost
for(i=0; i<N; i++)	c_2
arr[i] = 0;	c_1

$(N+1) \times c_2 + N \times c_1 = (c_2 + c_1) \times N + c_2$

- Both algorithms are of the same order: *O(N)*

Example (cont'd)

Algorithm 3	***Cost***
sum = 0;	c_1
for(i=0; i<N; i++)	c_2
for(j=0; j<N; j++)	c_2
sum += arr[i][j];	c_3

$c_1 + c_2 \times (N+1) + c_2 \times N \times (N+1) + c_3 \times N^2 = O(N^2)$

Example #1: carry n books from one bookshelf to another one

- How many operations?
- n pick-ups, n forward moves, n drops and n reverse moves 4 n operations
- 4n operations = c. n = O(c. n) = O(n)
- Similarly, any program that reads n inputs from the user will have minimum time complexity O(n).

Example #2: Locating Roll-Number record in Attendance Sheet

What is the time complexity of search?

- Binary Search algorithm at work
  - O(log n)
- Sequential search?
  - O(n)

Example #3: Teacher of CSE 221 gives gifts to first 10 students

- There are n students in the queue.
- Teacher brings one gift at a time.
- Time complexity = O(c. 10) = O(1)
- Teacher will take exactly same time irrespective of the line length.

Loops with Break

```
for (j = 0; j < n; ++j)
{
 // 3 atomics
 if (condition) break;
}
```

- Upper bound = O(4n) = O(n)
- Lower bound = $\Omega(4) = \Omega(1)$
- Complexity = O(n)

Sequential Search

- Given an unsorted vector/list a[ ], find the location of element X.

```
for (i = 0; i < n; i++) {
        if (a[i] == X) return true;
}
return false;
```

- Input size: n = array size()
- Complexity = O(n)

If-then-else Statement

```
if(condition)
    i = 0;
else
    for ( j = 0; j < n; j++)
        a[j] = j;
```

- Complexity = ??

= O(1) + max (O(1), O(N))

= O(1) + O(N)

= O(N)

Consecutive Statements

```
for (j = 0; j < n; ++j) {
    // 3 atomics
}
for (j = 0; j < n; ++j) {
    // 5 atomics
}
```

- Add the complexity of consecutive statements
- Complexity = O(3n + 5n) = O(n)

Nested Loop Statements

- Analyze such statements inside out

```
for (j = 0; j < n; ++j) {
   // 2 atomics
   for (k = 0; k < n; ++k) {
      // 3 atomics
   }
}
```

Example

- **Code:**

```
a = b;
```

- **Complexity:**

O(1)

Example

Code:

```
sum = 0;
for (i=1; i <=n; i++)
    sum += n;
```

- Complexity:

  O(n)

Example

- **Code:**

```
sum1 = 0;
for (i=1; i<=n; i++)
    for (j=1; j<=n; j++)
        sum1++;
```

- **Complexity:**

O(n^2)

Example

- **Code:**

```
sum2 = 0;
for (i=1; i<=n; i++)
    for (j=1; j<=i; j++)
        sum2++;
```

- **Complexity:**

O(n^2)

Example

- Code:

```
sum = 0;
  for (j=1; j<=n; j++)
     for (i=1; i<=j; i++)
        sum++;
  for (k=0; k<n; k++)
     A[k] = k;
```

- Complexity:

O(n^2)

Example

- **Code:**

```
sum1 = 0;
for (k=1; k<=n; k*=2)
   for (j=1; j<=n; j++)
      sum1++;
```

- **Complexity:**

  O(nlogn)

Example

- **Code:**

```
sum2 = 0;
for (k=1; k<=n; k*=2)
    for (j=1; j<=k; j++)
        sum2++;
```

- **Complexity:** O(n)

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \ldots + x^n = \frac{x^{n+1} - 1}{x - 1} (x \neq 1)$$

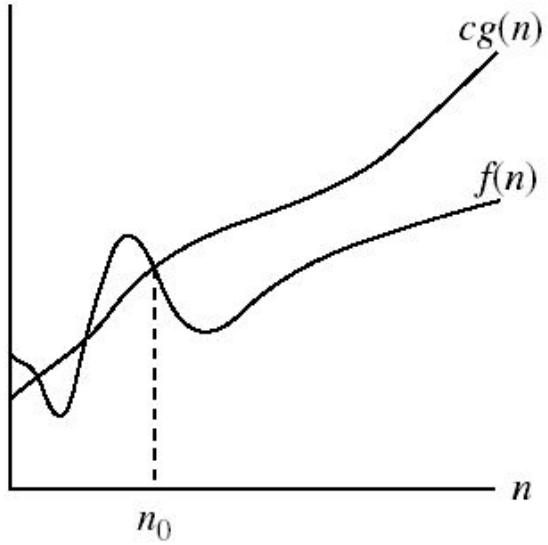
Classifying functions by their Asymptotic Growth Rates

1. O(g(n)), Big-Oh of g of n, the Asymptotic Upper Bound
2. Θ(g(n)), Theta of g of n, the Asymptotic Tight Bound
3. Ω(g(n)), Omega of g of n, the Asymptotic Lower Bound

Asymptotic notations

- *O-notation*

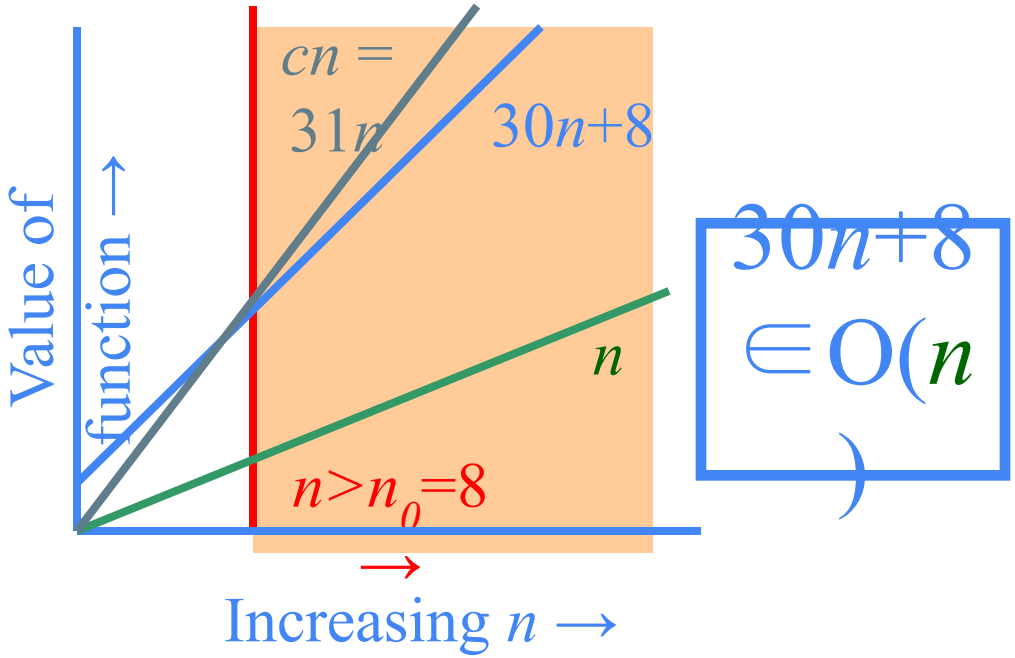
$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\} .$$



$g(n)$ is an ***asymptotic upper bound*** for $f(n)$.

Big-O example, graphically

- Note 30*n*+8 isn't less than *n* *anywhere* (*n*>0).
- It isn't even less than 31*n* *everywhere*.
- But it *is* less than 31*n* <u>everywhere to the right of *n*=8</u>.



Big-O

$f(n) = O(g(n))$: there exist positive constants c and n_0 such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$

- What does it mean?
  - If $f(n) = O(n^2)$, then:
    - $f(n)$ can be larger than n^2 sometimes, **but...**
    - We can choose some constant ***c*** and some value n_0 such that for **every** value of ***n*** larger than $\boldsymbol{n_0}$: $f(n) < cn^2$
    - That is, for values larger than n_0, $f(n)$ is never more than a constant multiplier greater than n^2
    - Or, in other words, $f(n)$ does not grow more than a constant factor faster than n^2.

Examples

- $2n^2 = O(n^3)$: $2n^2 \leq cn^3 \Rightarrow 2 \leq cn \Rightarrow c = 1$ and $n_0 = 2$
- $n^2 = O(n^2)$: $n^2 \leq cn^2 \Rightarrow c \geq 1 \Rightarrow c = 1$ and $n_0 = 1$
- $1000n^2+1000n = O(n^2)$:

$1000n^2+1000n \leq 1000n^2+ n^2 = 1001n^2 \Rightarrow c=1001$ and $n_0 = 1000$

- $n = O(n^2)$: $n \leq cn^2 \Rightarrow cn \geq 1 \Rightarrow c = 1$ and $n_0 = 1$

More Big-O

$$20n^2 + 2n + 5 = O(n^2)$$

- Prove that:
- Let $c = 21$ and $n_0 = 4$
- $21n^2 > 20n^2 + 2n + 5$ for all $n > 4$

  $n^2 > 2n + 5$ for all $n > 4$

  TRUE

Examples

- Show that

$$2^n + n^2 = O(2^n)$$

-

$$2 \times 2^n > 2^n + n^2$$
$$2^{n+1} > 2^n + n^2$$
$$2^{n+1} - 2^n > n^2$$
$$2^n(2-1) > n^2$$
$$2^n > n^2 \quad \forall n \geq 5$$

More Examples

- Show that $30n+8$ is $O(n)$.
  - Show $\exists\, c, n_0$: $30n+8 \leq cn$, $\forall\, n > n_0$.
    - Let $c=31$, $n_0=8$. Assume $n>n_0=8$. Then $cn = 31n = 30n + n > 30n+8$, so $30n+8 < cn$.

No Uniqueness

- There is no unique set of values for n_0 and c in proving the asymptotic bounds
- Prove that $100n + 5 = O(n^2)$
  - $100n + 5 \leq 100n + n = 101n \leq 101n^2$

    for all $n \geq 5$

    $n_0 = 5$ and $c = 101$ is a solution
  - $100n + 5 \leq 100n + 5n = 105n \leq 105n^2$

    for all $n \geq 1$

    $n_0 = 1$ and $c = 105$ is also a solution

Must find **SOME** constants c and n_0 that satisfy the asymptotic notation relation

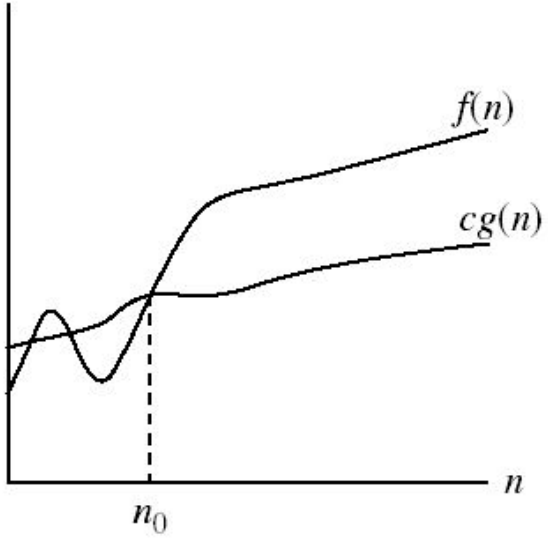
Big-O

$$2n^2 = O(n^2)$$
$$1{,}000{,}000n^2 + 150{,}000 = O(n^2)$$
$$5n^2 + 7n + 20 = O(n^2)$$
$$2n^3 + 2 \neq O(n^2)$$
$$n^{2.1} \neq O(n^2)$$

Asymptotic notations

- Ω - *notation*

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0\} .$$



$\Omega(g(n))$ is the set of functions with larger or same order of growth as $g(n)$

$g(n)$ is an ***asymptotic lower bound*** for $f(n)$.

Examples

- $5n^2 = \Omega(n)$

  $\exists\ c, n_0$ such that: $0 \le cn \le 5n^2 \Rightarrow cn \le 5n^2 \Rightarrow c = 5$ and $n_0 = 1$

- $100n + 5 \neq \Omega(n^2)$

  $\exists\ c, n_0$ such that: $0 \le cn^2 \le 100n + 5$

  $100n + 5 \le 100n + 5n\ (\forall\ n \ge 1) = 105n$

  $cn^2 \le 105n \Rightarrow n(cn - 105) \le 0$

  Since n is positive $\Rightarrow cn - 105 \le 0 \Rightarrow n \le 105/c$

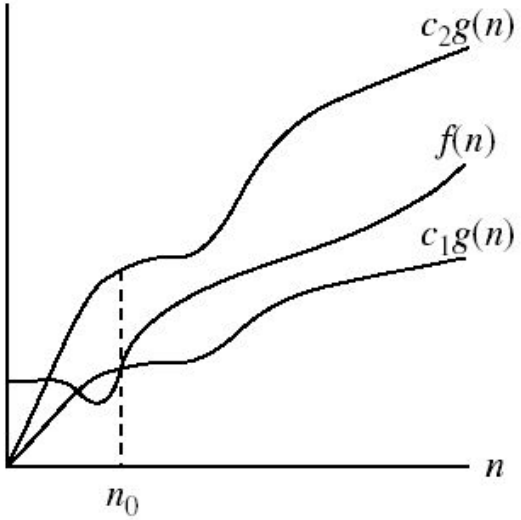
  $\Rightarrow$ contradiction: n cannot be smaller than a constant

- $2n = \Omega(n)$, $n^3 = \Omega(n^2)$, $n = \Omega(\log n)$

Asymptotic notations (cont.)

- *Θ-notation*

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0\} .$$



Θ(*g*(n)) is the set of functions with the same order of growth as *g*(n)

$g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

Examples

- Prove that:

$$20n^3 + 7n + 1000 = \Theta(n^3)$$

- Let $c_1 = 21$ and $n_0 = 10$
- $21n^3 > 20n^3 + 7n + 1000$ for all $n > 10$

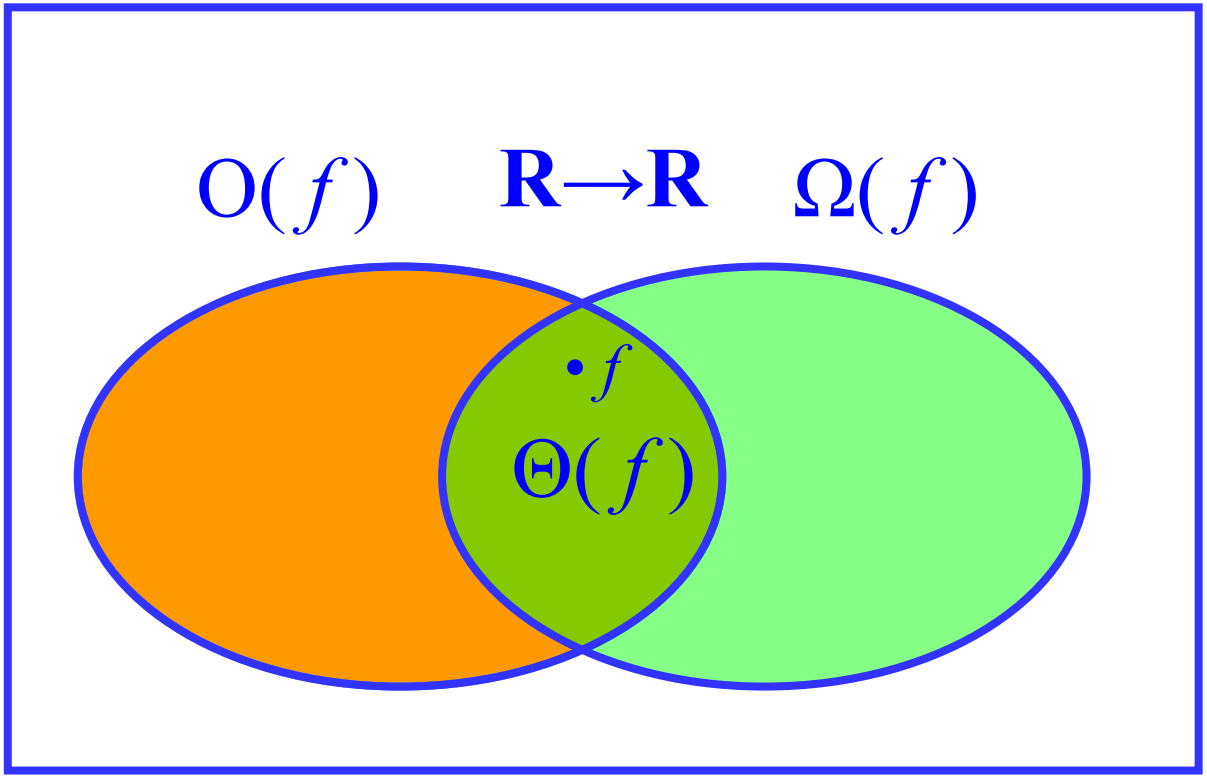
  $n^3 > 7n + 1000$ for all $n > 10$

  TRUE, but we also need...
- Let $c_2 = 20$ and $n_0 = 1$
- $20n^3 < 20n^3 + 7n + 1000$ for all $n \geq 1$

  TRUE

Relations Between Different Sets

- Subset relations between order-of-growth sets.



Logarithms and properties

- In algorithm analysis we often use the notation "log n" without specifying the base

Binary logarithm $\lg n = \log_2 n$

Natural logarithm $\ln n = \log_e n$

$\lg^k n = (\lg n)^k$

$\lg \lg n = \lg(\lg n)$

$\log x^y = y \log x$

$\log xy = \log x + \log y$

$\log \frac{x}{y} = \log x - \log y$

$a^{\log_b x} = x^{\log_b a}$

$\log_b x = \frac{\log_a x}{\log_a b}$

Simplifying Assumptions

1. If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$

2. If $f(n) = O(kg(n))$ for any $k > 0$, then $f(n) = O(g(n))$

3. If $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$,
   then $f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$

4. If $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$,
   then $f_1(n) * f_2(n) = O(g_1(n) * g_2(n))$

Some Simplified Rules

- $O(1) = c$, where c is a constant
- $O(n) = c*n = cn$, where c is constant and n is variable
- $c_1*O(1) = c_1*c = c_2 = O(1)$, where c, c_1, c_2 are constants
  - $O(1) + O(1) + O(1) = 3*O(1) = O(1)$
  - $5*O(1) = O(1)$
- $n*O(1) = n*c = cn = O(n)$, where c is constant and n is variable
- $O(m) + O(n) \neq O(m+n)$
- $O(m) * O(n) = c_1 m c_2 n = (c_1*c_2)(mn) = (c_2)(mn) = O(mn)$
- $O(m)*O(n)*O(p)*O(q) = O(m(n(p(q)))) = O(mnpq)$
  - Example nested for loops
- $O(an^2 + bn + c) = O(n^2)$ where a, b , c are constants

Common Summations

- Arithmetic series:

$$\sum_{k=1}^{n} k = 1 + 2 + \ldots + n = \frac{n(n+1)}{2}$$

- Geometric series:

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \ldots + x^n = \frac{x^{n+1} - 1}{x - 1} \left(x \neq 1\right)$$

  - Special case: $|x| < 1$:

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}$$

- Harmonic series:

$$\sum_{k=1}^{n} \frac{1}{k} = 1 + \frac{1}{2} + \ldots + \frac{1}{n} \approx \ln n$$

- Other important formulas:

$$\sum_{k=1}^{n} \lg k \approx n \lg n$$

$$\sum_{k=1}^{n} k^p = 1^p + 2^p + \ldots + n^p \approx \frac{1}{p+1} n^{p+1}$$

Summary

- Time complexity is a measure of algorithm efficiency
- Efficient algorithm plays the major role in determining the running time.

Q: Is it possible to determine running time based on algorithm's time complexity alone?

- Minor tweaks in the code can cut down the running time by a factor too.
- Other items like CPU speed, memory speed, device I/O speed can help as well.
- For certain problems, it is possible to allocate additional space & improve time complexity.